DynamicServiceOrchestration

System Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic.

DynamicServiceOrchestration - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/DynamicServiceOrchestration. Entry point: ./cmd/serviceorchestration-dynamic. Module: arrowhead/ah5-core-go/serviceorchestration-dynamic. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8441 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8441". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

1 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-dynamic.

DynamicServiceOrchestration (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary serviceorchestration-dynamic by the arrowhead-core-rust crate in ... serviceorchestration-dynamic.json and documentation/ link to ../examples and ../documentation/DynamicServiceOrchestration.

Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysDD.tex

- Go: Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic/documentation/service-design/DynamicServiceOrchestration-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/DynamicServiceOrchestration/DynamicServiceOrchestration-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

DynamicServiceOrchestration

System Design Description

Abstract
This document describes the design and implementation of the DynamicServiceOrchestration core system of Arrowhead 5.2.2 in Java Spring (`ah5-core-java-spring/serviceorchestration-dynamic`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 5
2.5 Local modifications 6
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **10**
5.1 Amendments 10
5.2 Quality Assurance 10

1 Overview

This document describes the DynamicServiceOrchestration system implementation, which realizes the DynamicServiceOrchestration SysD [1]. DynamicServiceOrchestration finds, at request time, the providers a consumer shall use. It discovers candidates in the ServiceRegistry, keeps only the authorized, not blacklisted and not locked ones with matching interfaces, applies preferences, QoS requirements, matchmaking and exclusivity, obtains the authorization tokens the providers require, and returns the result; it also pushes results to subscribers. In this workspace it extends orchestration to neighbour clouds through the Gatekeeper, and enforces the active-plant consumer/service scopes submitted by PlantDescriptionEngine. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation is based on the SysD document DynamicServiceOrchestration [1], the upstream Eclipse Arrowhead Java Spring implementation 5.2.2 [2] and the local changes listed in Section 2.5.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9 (web, security, data JPA, Quartz), springdoc-openapi 2.8 (Swagger UI), Log4j 2.
- Maven module `serviceorchestration-dynamic` of `ah5-core-java-spring`; it depends on `arrowhead-common-utils` and `arrowhead-data-transfer-objects` 5.2.2 from the local `ah5-common-java-spring` (Eclipse Paho MQTT v3 1.2.5, RabbitMQ AMQP client 5.22.0).
- Database: MariaDB/MySQL (MariaDB Connector/J; MySQL 8.4 server in the test cloud), schema validated by Hibernate (`ddl-auto=validate`); SQL scripts in `src/main/resources/database`.
- Stateless request processing; all state is in the database. Scheduled jobs use Quartz.

2.2 Functional properties implementation

The Java packages below `eu.arrowhead` implement the system:

Package	Responsibility
`api.http, api.mqtt`	Controllers and topic handlers of the five services.
`service.utils.LocalService Orchestration`	The local orchestration pipeline.
`service.utils.InterCloud ServiceOrchestration`	Inter-cloud orchestration through the Gatekeeper (local extension).
`service.utils.matchmaker`	Selection of one candidate (`DefaultServiceInstance Matchmaker`).
`driver.QoSDriver`	QoS evaluation through DeviceQoSEvaluator.
`service.thread`	Push orchestration thread and workers.
`service, service.model, service.validation`	Orchestration, push, lock and history logic; request validation and normalization.
`jpa, quartz`	Job, subscription, lock, QoS result and log entities; cleaner job.

Every request passes the common filter chain: authentication filter (by `authentication.policy`), optional blacklist filter, management service filter (for management services), then the controller, validation and normalization, the service logic and the database service. The same logic is reachable through the MQTT topic handlers.

2.2.1 Data handled and stored

Database `ah_serviceorchestration_dynamic`:

Tables	Content
`orchestration_job`	Pull and push jobs with status and message.
`subscription`	Push subscriptions with request and notify interface.
`orchestration_lock`	Exclusive and temporary locks.
`qos_eval_result`	Results of QoS evaluations.
`logs`	Log entries for `get-log`.

The data models of the interfaces are the Java records of `eu.arrowhead.dto` (common library); the IDD documents name them without the `DTO` suffix. The records used by this system are: `AuthorizationTokenGenerationResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponseDTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `OrchestrationHistoryQueryRequestDTO`, `OrchestrationHistoryResponseDTO`, `OrchestrationJobDTO`, `OrchestrationLockListRequestDTO`, `OrchestrationLockListResponseDTO`, `OrchestrationLockQueryRequestDTO`, `OrchestrationLockRequestDTO`, `OrchestrationLockResponseDTO`, `OrchestrationNotifyInterfaceDTO`, `OrchestrationPlantProviderDTO`, `OrchestrationPlantRuleDTO`, `OrchestrationPlantRulesDTO`, `OrchestrationPushJobListResponseDTO`, `OrchestrationPushTriggerDTO`, `OrchestrationRequestDTO`, `OrchestrationResponseDTO`, `OrchestrationResultDTO`, `OrchestrationServiceRequirementDTO`, `OrchestrationSubscriptionListRequestDTO`, `OrchestrationSubscriptionListResponseDTO`, `OrchestrationSubscriptionQueryRequestDTO`, `OrchestrationSubscriptionRequestDTO`, `OrchestrationSubscriptionResponseDTO`, `PageDTO`, `QoSRequirementDTO`, `ServiceInstanceInterfaceResponseDTO`.

2.3 Non functional properties implementation

2.3.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore). The authentication filters of the common library implement the four authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking in Tomcat (both local). The private key of the keystore is also used to sign the system's outgoing requests where required.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.3.3 Internal monitoring

Log4j 2 logs to stdout and rolling files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The native `sysMonitor` publisher (`eu.arrowhead.common.sysmonitor`) publishes `echo`, `power` and `status` every 30 s.

2.3.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding, e.g. `SERVICE_DISCOVERY_POLICY`); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD. `generalManagement.get-config` returns the public ones.

2.4 Deployment

The test cloud (`Arrowhead-test-cloud-java`) builds the image `arrowhead-serviceorchestration-dynamic:5.2.2-sysmonitor` from the released image `aitiaiiot/arrowhead-serviceorchestration-dynamic:5.2.2` with the locally built `arrowhead-serviceorchestration-dynamic-5.2.2.jar` (`java-patch/Dockerfile`). The container runs read-only with the configuration baked into `/app/config`, only its keystores mounted, and publishes port 8441 on the loopback address of the host. Its database runs in a separate container.

2.5 Local modifications

Compared to upstream 5.2.2, this workspace changes:

- Startup is refused when the effective configuration is below the secure profile (`SecurityProfileGuard`).
- Inter-cloud orchestration through the Gatekeeper; `ALLOW_INTERCLOUD` falls back to it without local results, `ONLY_INTERCLOUD` goes straight to it. Warnings `interCloud`, `interCloudDisabled`, `interCloudFailed`.
- Job messages name the selected providers (`<n> local result: <providers>`, `<n> intercloud result: <provider>@<cloud>`).
- The HTTP `monitor` service (`/serviceorchestration/monitor/echo`) was removed; telemetry is the native `sysMonitor` service.
- `http.collector.bootstrap.services` also lists the authorization and blacklist services, which the orchestrator cannot orchestrate for itself.
- Token-to-certificate binding, CRL checking, MariaDB driver.

2.6 Verification

The module has 30 JUnit test classes; the regression report set recorded on 2026-11-07 covers 477 tests without failures. Live-cloud smoke tests are a separate verification step; the plant-management migration is not claimed as deployed by this update.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`serviceOrchestration`	[1]	[3]	[4], [5]
`serviceOrchestrationPush Management`	[1]	[6]	[7], [8]
`serviceOrchestrationLock Management`	[1]	[9]	[10], [11]
`serviceOrchestrationHistory Management`	[1]	[12]	[13], [14]
`serviceOrchestrationPlant Management`	[1]	[15]	[16]
`generalManagement`	[1]	[17]	[18], [19]
`sysMonitor`	[1]	[20]	[21]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[22]	[23]	–
identity	[24]	[25]	–
authorization; authorization Management	[26]	[27]	–
authorizationTokenManagement	[26]	[28]	–
blacklistDiscovery; blacklist Management	Blacklist	–	–
qualityEvaluation	DeviceQo SEvaluator	–	–
translationBridgeManagement	Translation Manager	–	–
globalServiceDiscovery; interCloud Negotiation	Gatekeeper	–	–
notify interface	Subscribers	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysD.pdf.

[2] AITIA International, “Eclipse arrowhead 5 core systems (java spring) and documentation,” https://aitia-iiot.github.io/ah5-docs-java-spring/, 2025.

[3] J. Delsing, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[4] ——, “serviceOrchestration HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.pdf.

[5] ——, “serviceOrchestration MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestration-MQTT-IDD.pdf.

[6] ——, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

[7] ——, “serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

[8] ——, “serviceOrchestrationPushManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-MQTT-IDD.pdf.

[9] ——, “serviceOrchestrationLockManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-SD.pdf.

[10] ——, “serviceOrchestrationLockManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-HTTP-IDD.pdf.

[11] ——, “serviceOrchestrationLockManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-MQTT-IDD.pdf.

[12] ——, “serviceOrchestrationHistoryManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

[13] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

[14] ——, “serviceOrchestrationHistoryManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-MQTT-IDD.pdf.

[15] ——, “serviceOrchestrationPlantManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPlantManagement-SD.pdf.

[16] ——, “serviceOrchestrationPlantManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPlantManagement-HTTP-IDD.pdf.

[17] ——, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

[18] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.pdf.

[19] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-MQTT-IDD.pdf.

[20] ——, “sysMonitor – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-sysMonitor-SD.pdf.

[21] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[22] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[23] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[24] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[25] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identity-SD.pdf.

[26] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[27] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

[28] ——, “authorizationTokenManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 System context

DynamicServiceOrchestration discovers providers through ServiceRegistry, verifies consumers through ConsumerAuthorization and returns eligible interfaces. PDE supplies engineering constraints through a separate snapshot API. The implementation shares the native Go core runtime; no Java or Rust process implements it. Existing supported dynamic orchestration and optional SimpleStore services retain their separate identities and state.

2 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains `serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.

1 DynamicServiceOrchestration

1.1 System overview

Native Rust implementation of the local Java module `serviceorchestration-dynamic/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

1.2 Produced and consumed services

Produced HTTPS definitions: `generalManagement`, `serviceOrchestration`, `serviceOrchestrationHistoryManagement`, `serviceOrchestrationLockManagement`, `serviceOrchestrationPlantManagement`, `serviceOrchestrationPushManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

1.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

1.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

1.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Local/inter-cloud orchestration, authorization, operation filtering, tokens, preferences, matchmaking, locks, history and HTTPS push are implemented. The local PDE extension adds atomic persistent plant scopes and ranked providers; managed scopes cannot escape through inter-cloud flags. Translation, QoS, complete blacklist pipelines, registry-event-driven reevaluation and full Java parity remain unqualified.

2 Implementation structure

See `src/security.rs`, `src/store.rs`, `src/web.rs` and the role-specific module. Transactions clone state and publish only after durable rename. State locking prevents concurrent processes from sharing the same path. A directory-sync failure after rename is an uncertain durable commit, not an automatic rollback. HTTPS push callbacks are at-least-once across crash recovery; downstream effects can require reconciliation.

2.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceorchestration/general/mgmt/get-config	management
POST	/serviceorchestration/general/mgmt/logs	management
POST	/serviceorchestration/orchestration/pull	identity
POST	/serviceorchestration/orchestration/subscribe	identity
DELETE	/serviceorchestration/orchestration/unsubscribe/{id}	identity
POST	/serviceorchestration/orchestration/mgmt/push/subscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/trigger	management
DELETE	/serviceorchestration/orchestration/mgmt/push/unsubscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/query	management
POST	/serviceorchestration/orchestration/mgmt/history/query	management
POST	/serviceorchestration/mgmt/plant-rules/query	management
PUT	/serviceorchestration/mgmt/plant-rules/replace	management
POST	/serviceorchestration/orchestration/mgmt/lock/create	management
POST	/serviceorchestration/orchestration/mgmt/lock/query	management
DELETE	/serviceorchestration/orchestration/mgmt/lock/remove/{owner}	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

3 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains

`serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.